ASP.NET

Piyush Pant

ASP.NET

ASP.NET is Microsoft's framework for building dynamic web applications, websites, and services.

It comes in two main forms:

ASP.NET (Classic Framework)

- Introduced in **2002** as part of the .NET Framework.
- Runs only on **Windows** and is closely tied to IIS (Internet Information Services).
- Includes **Web Forms, MVC, and Web API** for different styles of development.
- Still used in many legacy enterprise applications.

ASP.NET Core (Modern Version)

- Released in **2016** as a complete redesign.
- **Cross-platform** (Windows, Linux, macOS).
- Open-source, modular, and optimized for **cloud, microservices, and high performance**.
- Supports **MVC, Razor Pages, Blazor, and minimal APIs**.
- Actively developed and recommended for new projects.

ASP.NET (Active Server Pages .NET)

ASP.NET, developed by Microsoft, is a popular framework for building web applications. It provides different models for web development, each suited to various needs. The three main frameworks under the ASP.NET umbrella are:

ASP.NET Web Forms

A traditional event-driven programming model that uses a drag-and-drop approach with server controls.

Features:

- Supports Rapid Application Development (RAD) with a visual designer.
- Uses ViewState to maintain state across postbacks.
- Heavily reliant on server-side controls.
- Ideal for enterprise applications that require a quick UI development approach.

Use Case: Suitable for developers who prefer a Windows Forms-like experience for web development.

ASP.NET (Active Server Pages .NET)

ASP.NET MVC (Model-View-Controller)

A lightweight, testable, and highly structured framework that follows the MVC pattern.

Features:

- Separates application logic into Model (data), View (UI), and Controller (handles requests).
- Promotes clean architecture and testability.
- Uses convention-based routing instead of event-driven development.
- Offers full control over HTML, CSS, and JavaScript.

Use Case: Best suited for complex applications requiring high maintainability, testability, and scalability.

ASP.NET Web Pages

Overview: A lightweight framework for building simple web pages using Razor syntax.

Features:

- Uses Razor markup for embedding C# code into HTML.
- Simplifies development with minimal setup.
- Suitable for small-scale applications and rapid prototyping.

Use Case: Ideal for developers who need to create simple, content-focused web pages quickly.

Framework/Model	Platform	Best Use Case
Web Forms	ASP.NET Framework	Legacy enterprise apps
MVC	Both	Structured web apps
Web API	Both	RESTful services
Core MVC	ASP.NET Core	Modern, scalable web apps
Razor Pages	ASP.NET Core	Simple page-focused apps
Blazor	ASP.NET Core	Interactive UIs with C#
Minimal APIs	ASP.NET Core	Microservices, lightweight APIs

ASP.NET

ASP.NET (Framework)

- Released in **2002** as part of the .NET Framework.
- Runs only on **Windows** and is tightly integrated with IIS (Internet Information Services).
- Includes **Web Forms, MVC, and Web API** models.
- Still used in many legacy enterprise applications.

ASP.NET Core

- Released in **2016** as a complete redesign.
- **Cross-platform** (Windows, Linux, macOS).
- Open-source, modular, and optimized for **cloud and microservices**.
- Supports **MVC, Razor Pages, Blazor, and minimal APIs**.
- Actively developed and recommended for new projects.
- When people say **ASP.NET** today, they often mean **ASP.NET Core**, since it's the current and future direction of Microsoft's web development platform. The older ASP.NET Framework is still supported but mainly for maintaining existing applications.

ASP.NET VS ASPNET Core

	ASP.NET	ASP.NET Core
Platform	Windows-only; built on the .NET Framework.	Cross-platform; built on the modern .NET Core framework.
Hosting	Can only be hosted on IIS.	Flexible hosting options: IIS, Nginx, Apache, or self-hosted.
Performance	Performance is good but constrained by older architecture.	Higher performance due to lightweight architecture and optimizations.
Architecture	Monolithic; tightly coupled components.	Modular; allows lightweight and flexible configurations.
Open Source	Partially open source.	Fully open source and developed with community input.
Dependency Injection	Requires third-party tools for implementation.	Built-in dependency injection support.
Use Cases	Suitable for legacy applications.	Ideal for modern applications, cloud solutions, and microservices.

MVC (Model-View-Controller)

MVC is a software design pattern commonly used in .NET for developing web applications. It helps in separating the concerns of an application by dividing it into three main components: **Model**, **View**, and **Controller**. This architecture makes applications more maintainable and scalable, and it allows multiple developers to work on different aspects of the application independently.

Model

- The **Model** represents the data or business logic of the application.
- It encapsulates the data, typically coming from a database, and the logic for data manipulation, validation, and processing.
- The **Model** is responsible for retrieving data, saving data, and implementing business rules.
- It can be a class or set of classes, representing domain entities like Customer, Order, etc.

MVC (Model-View-Controller)

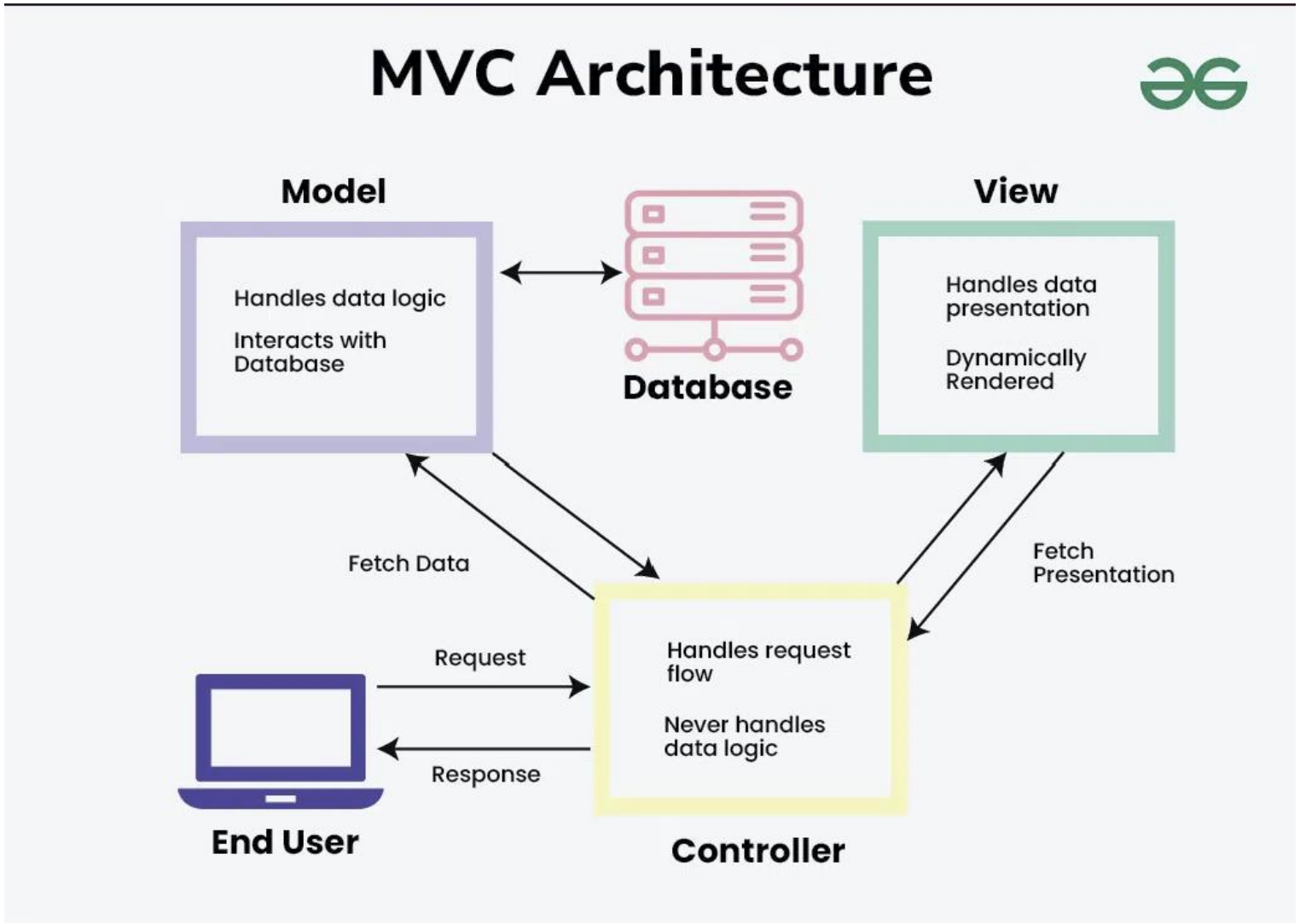
View

- The **View** is responsible for displaying the user interface (UI) of the application.
- It renders the data from the **Model** in a format that is suitable for the user to view.
- In web applications, the **View** could be HTML, CSS, or Razor Views (if using ASP.NET).
- The **View** does not directly contain logic to handle data, but it formats and presents the data provided by the **Controller**.

Controller

- The **Controller** acts as the intermediary between the **Model** and **View**.
- It handles user input (like button clicks, form submissions) and makes decisions based on that input.
- The **Controller** retrieves data from the **Model** and updates the **View** accordingly.
- It contains the application flow logic and communicates with the **Model** to process requests..

MVC Architecture



Routing in ASP.NET

In ASP.NET, routing is the mechanism that maps incoming URL requests to the appropriate controller actions. There are two main approaches: ***Conventional Routing*** (pattern-based, defined centrally) and ***Attribute Routing*** (defined directly on controllers/actions).

Conventional Routing

Routes are defined in a central place (e.g., `Program.cs` or `Startup.cs` in ASP.NET Core).

Pattern-based: Uses templates like `{controller}/{action}/{id?}`

Routing

Advantages:

- Centralized control of routes.
- Easy to apply consistent patterns across the app.

Limitations:

- Less flexible for complex or RESTful APIs.
- Requires sticking to predefined URL structures

Attribute Routing

Routes are defined directly on controllers and actions using attributes.

```
[Route("products")]
public class ProductsController : Controller
{
    [Route("details/{id}")]
    public IActionResult Details(int id) => View();
}
```

Attribute Routing

Advantages:

- Fine-grained control over individual routes.
- Ideal for RESTful APIs where URLs need to be descriptive and customized.
- Supports route constraints, prefixes, and tokens.

Limitations:

- Routes are scattered across controllers, making them harder to manage in large apps.

ASPECT	CONVENTIONAL ROUTING	ATTRIBUTE ROUTING
Definition Location	Centralized in `Program.cs`/`Startup.cs`	On controllers/actions via attributes
Flexibility	Limited, follows predefined patterns	High, allows custom URL design
Best Use Case	Simple apps, consistent URL structures	RESTful APIs, complex/custom routes
Maintainability	Easier for uniform apps	Better for APIs but harder in large apps

URL Routing (Convention)

URL Routing in .NET MVC is the process of mapping incoming requests (URLs) to appropriate controller actions. It defines the pattern in which the URLs should be structured and specifies which **Controller** and **Action** should handle the request.

Key Concepts of URL Routing:

Controller:

- The **Controller** is responsible for handling incoming HTTP requests.
- It contains action methods that process the request and return a response.

Action:

- An **Action** is a method inside the controller that is executed when a particular route is matched.
- Actions typically return a **View** (HTML content) or data (JSON, XML, etc.).

Route:

- A **Route** is a URL pattern that the routing engine matches to determine which controller and action to invoke.
- A route is defined using a pattern with placeholders such as `{controller}`, `{action}`, and `{id}`.

Default Route:

- The **default route** is the fallback route used if no specific route is matched.
- It is typically configured to map the URL to a default controller (`HomeController`) and action (`Index`).

URL Routing

Syntax for Defining Routes:

Routes are configured in the `Program.cs` (or `RouteConfig.cs`).

A route is defined using `MapControllerRoute` or `MapDefaultControllerRoute`.

Example of Default Route in **.NET 8**:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

`{controller=Home}`: The default controller is `HomeController`.

`{action=Index}`: The default action is `Index`.

`{id?}`: `id` is an optional parameter.

URL Routing

Positional Parameters:

- Parameters are included in the URL as `{parameter}` and mapped to method parameters in the controller action.
- Example: `Products/Details/5` maps to `{controller=Products}/{action=Details}/{id}`

Optional Parameters:

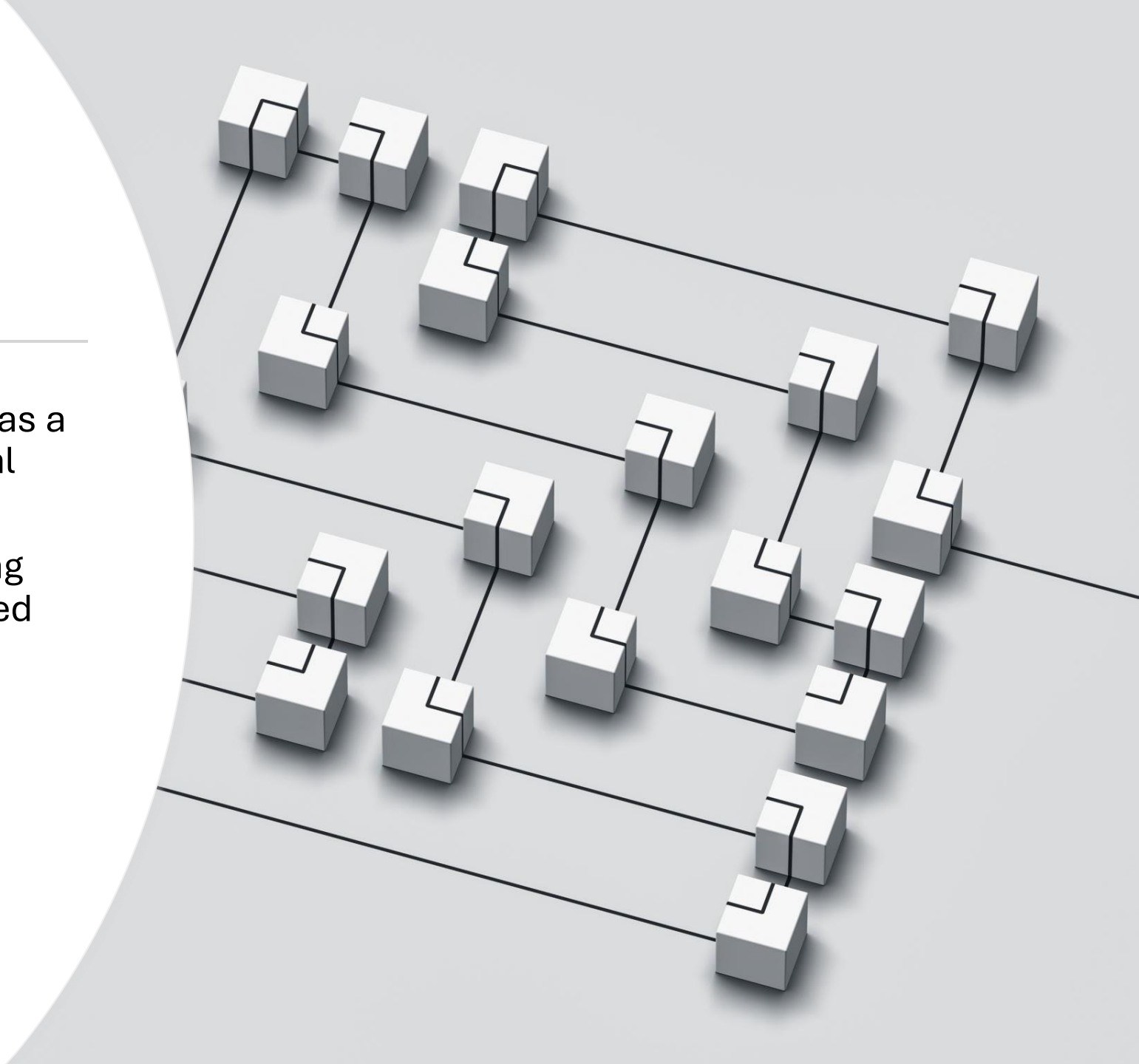
- Parameters can be made optional by using `UrlParameter.Optional`.
- Example: `{id?}` makes the `id` parameter optional.

- **Parameter Constraints**:
  - You can add constraints to parameters to restrict the values (e.g., only digits).
  - Example: `id:int` ensures `id` is an integer.

Note

Use **Conventional Routing** if your app has a consistent URL structure (e.g., traditional MVC websites).

- Use **Attribute Routing** if you're building **RESTful APIs** or need highly customized routes.
- In modern ASP.NET Core projects, developers often **combine both approaches**: conventional routing for general pages and attribute routing for APIs.



Routing

Razor Pages Routing

- Page-focused routing model.
- Routes are based on the folder and file structure (`/Pages` directory).
- Example: `Pages/Products/Details.cshtml` → `/Products/Details`.

Blazor Routing

- Used in Blazor apps (Server or WebAssembly).
- Routes are defined with the `@page` directive inside `.razor` components.

```
@page "/products/{id:int}"
```

Routing

Minimal API Routing

- Introduced in .NET 6 for lightweight APIs.
- Routes are defined directly in `Program.cs` using methods like `MapGet`, `MapPost`, etc.

```
app.MapGet("/hello", () => "Hello World!");
```

Custom Middleware Routing

- You can write your own middleware to intercept requests and route them manually.
- Useful for very specialized scenarios (e.g., custom URL rewriting).

Routing Use Case

Routing Type	Best Use Case
Conventional Routing	Traditional MVC apps with consistent patterns
Attribute Routing	RESTful APIs, custom routes per action
Endpoint Routing	Unified routing across MVC, Razor, SignalR
Razor Pages Routing	Page-focused apps with simple structure
Blazor Routing	Component-based apps using C# instead of JS
Minimal API Routing	Lightweight microservices and APIs
Custom Middleware	Specialized routing or URL rewriting

Authorization and Authentication

Authentication and **Authorization** are key security steps that ensure only the right users access a system and can perform permitted actions. Authentication verifies who you are, and Authorization determines what you can do.

- **Authentication**: Confirms the user's identity.
- **Authorization**: Determines what the verified user is allowed to access or perform.

Authentication

Authentication is the process of verifying the identity of a user or system. It ensures that the user is legitimate by validating credentials like passwords, OTPs, or biometrics.

Working

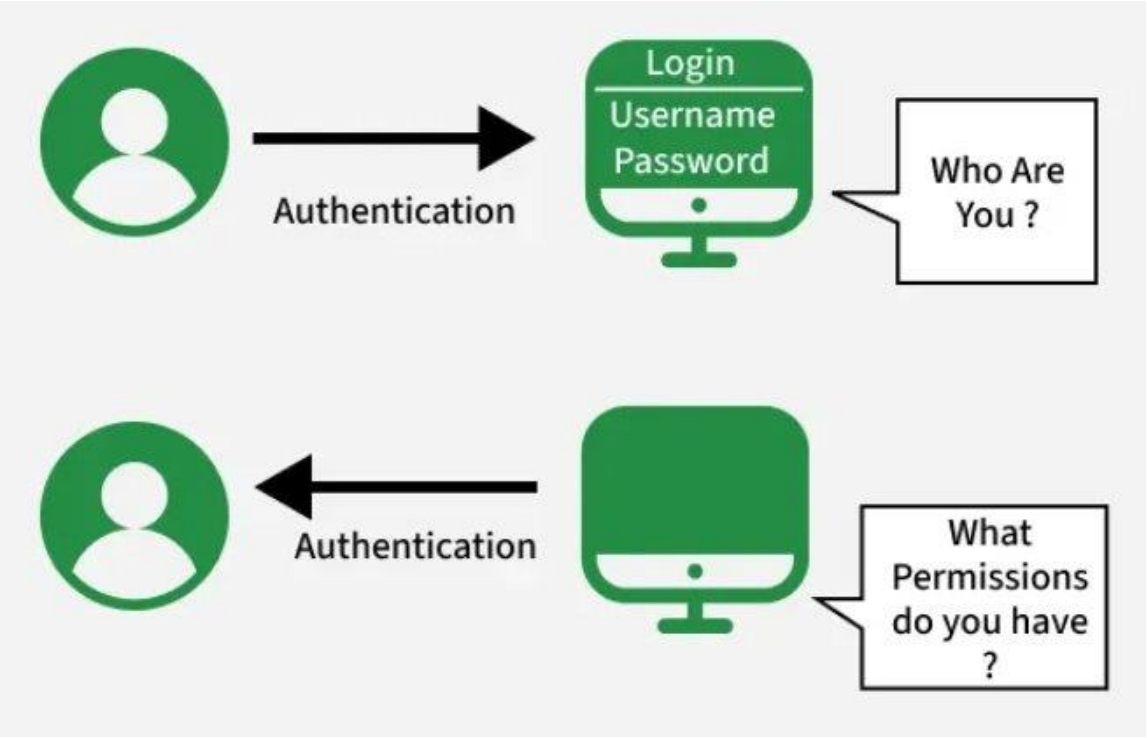
- The authentication process verifies the identity of a user before granting access to the system.
- User enters credentials (password, OTP, biometrics)
- System verifies the credentials
- If valid, the user is successfully authenticated

Authorization

Authorization determines the access rights and permissions of an authenticated user. It decides what resources the user can access and what actions they are allowed to perform.

Working

- The system uses authentication and authorization processes to control access and ensure security.
- System checks the user's roles or permissions
- Grants or denies access to resources
- Ensures the user can perform only allowed actions



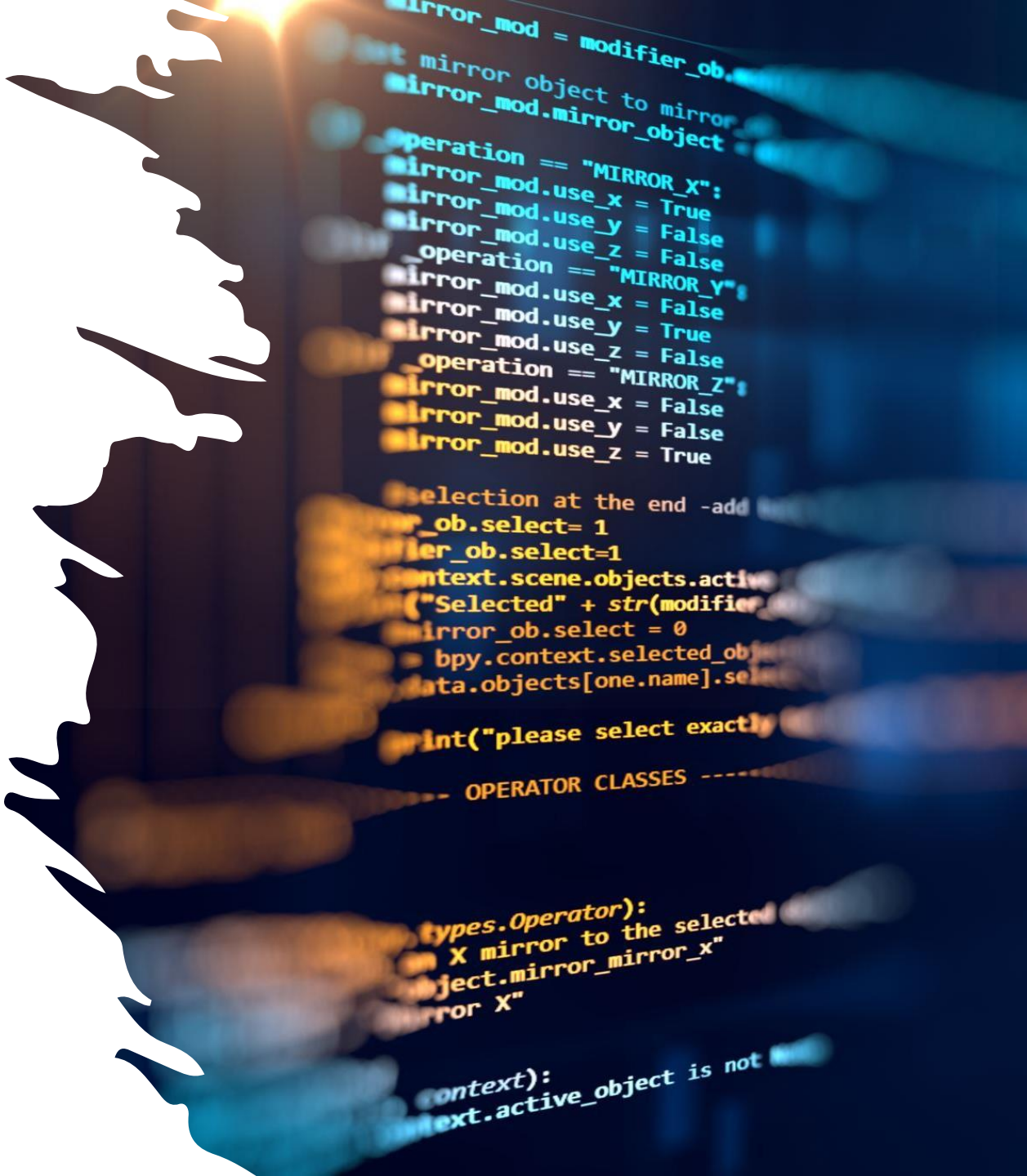
Authentication	Authorization
Verifies who the user is	Determines what the user can access
Performed before authorization	Happens after authentication
Requires login details (username, password, biometrics)	Requires user roles, privileges, or access levels
Determines if the user is valid	Determines what permissions the valid user has
Uses ID Tokens	Uses Access Tokens
Governed by OpenID Connect (OIDC)	Governed by OAuth 2.0
Credentials can be changed by the user	Permissions can only be changed by the system owner
Visible to the user (entering credentials)	Not visible to the user (handled in the background)
Examples: Password, OTP, fingerprint, face recognition	Examples: Admin rights, read/write access, role-based permissions

Middleware

- Middleware is **software that sits in the request pipeline** of an ASP.NET Core application.
- Every HTTP request passes through a **chain of middleware components** before reaching your endpoint (controller, Razor page, API, etc.).

Each middleware can:

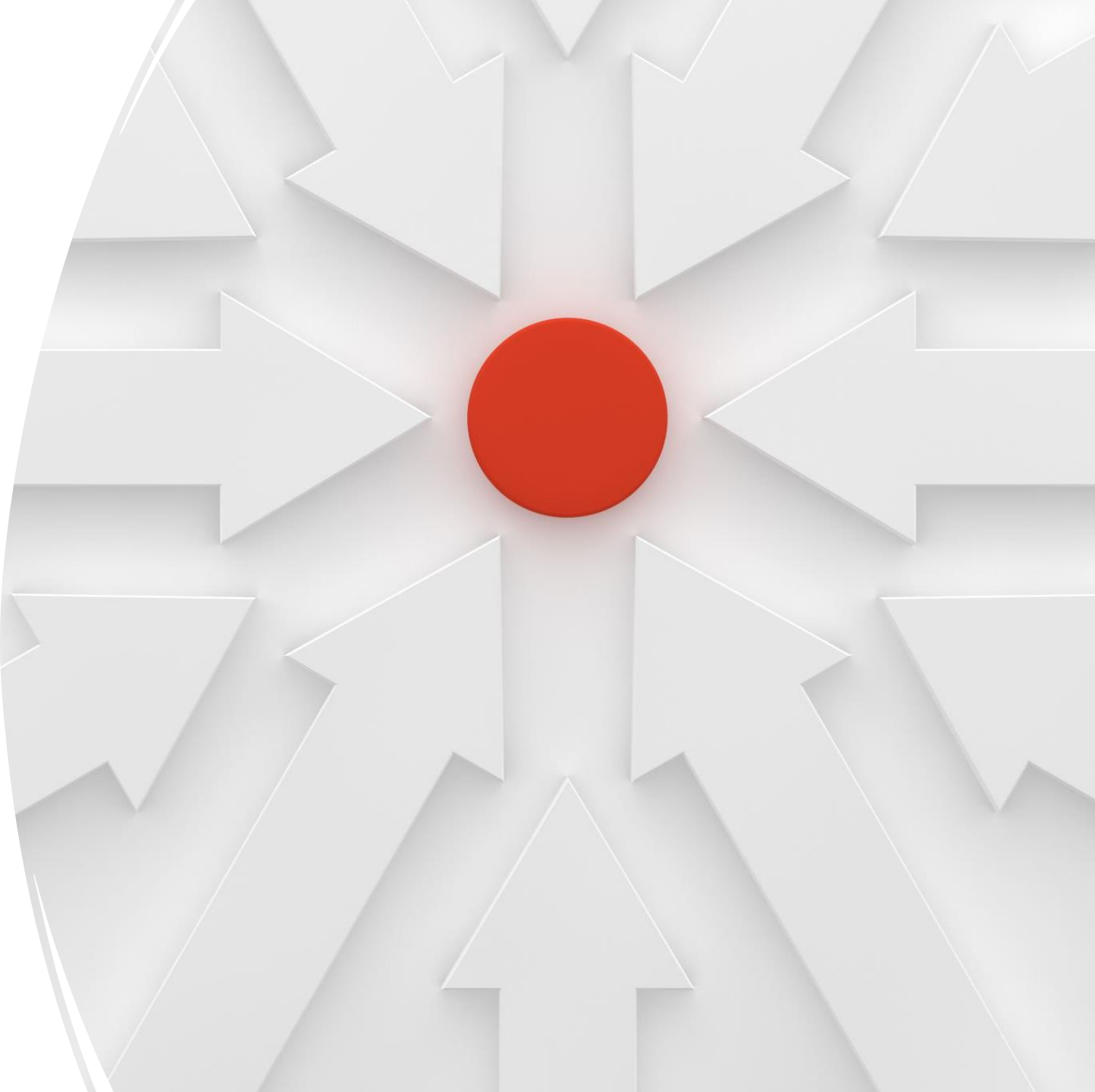
- **Do something with the request** (like logging, authentication, serving static files).
- **Pass the request to the next middleware** in the pipeline.
- **Do something with the response** before it goes back to the client.



Middleware Flow

Think of it like a series of checkpoints:

- Request enters the app.
- Middleware A runs → passes request forward.
- Middleware B runs → passes request forward.
- Middleware C runs → endpoint executes.
- Response flows back through C → B → A, each can modify it.



Common Built-in Middleware

UseRouting → matches requests to endpoints.

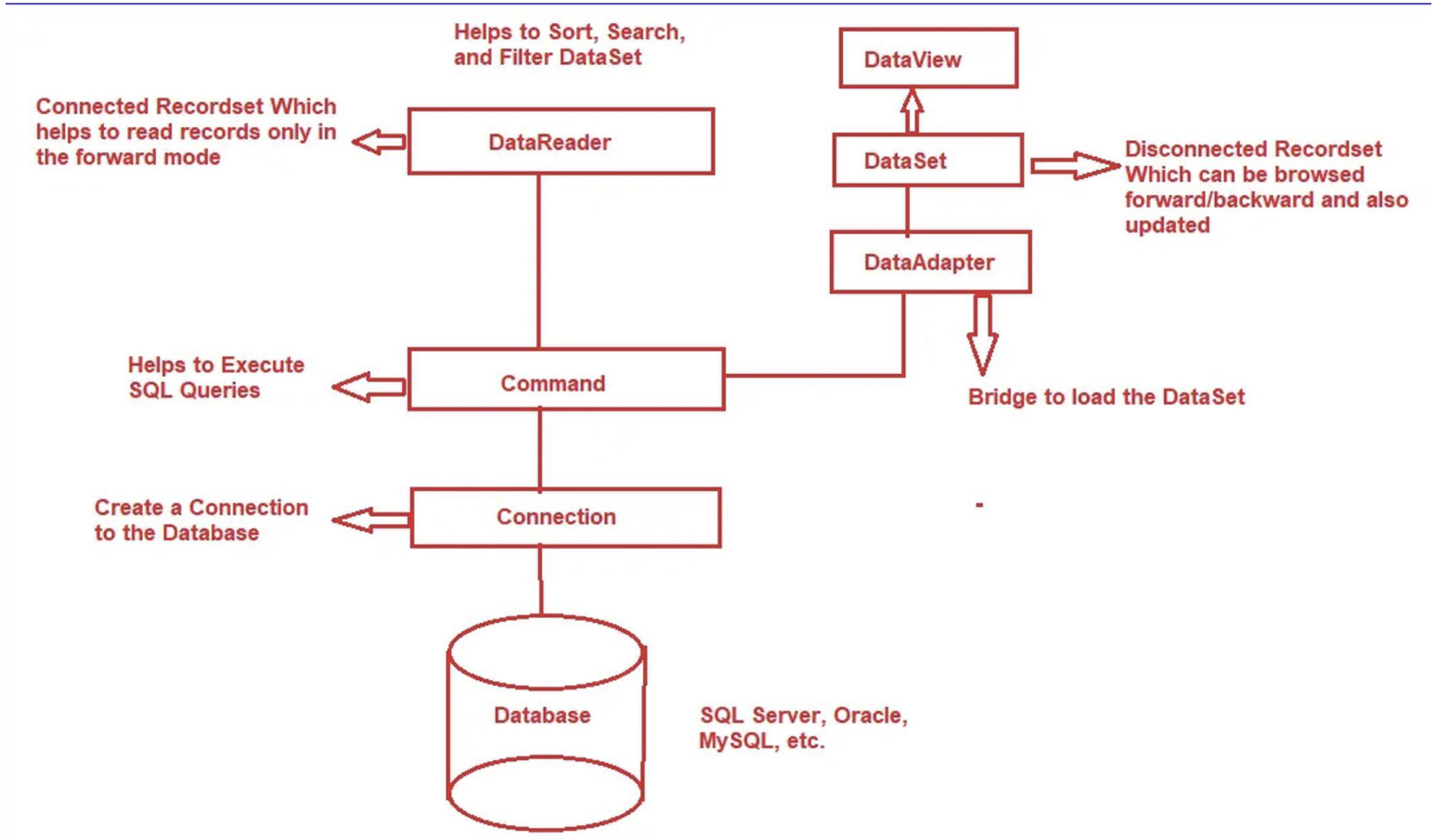
UseAuthentication → checks who the user is.

UseAuthorization → enforces what they can do.

UseStaticFiles → serves CSS, JS, images.

UseExceptionHandler → handles errors gracefully.

Order matters: authentication must run before authorization, for example.

Read More

ADO.NET Architecture